

The Threading Bits in utils.py

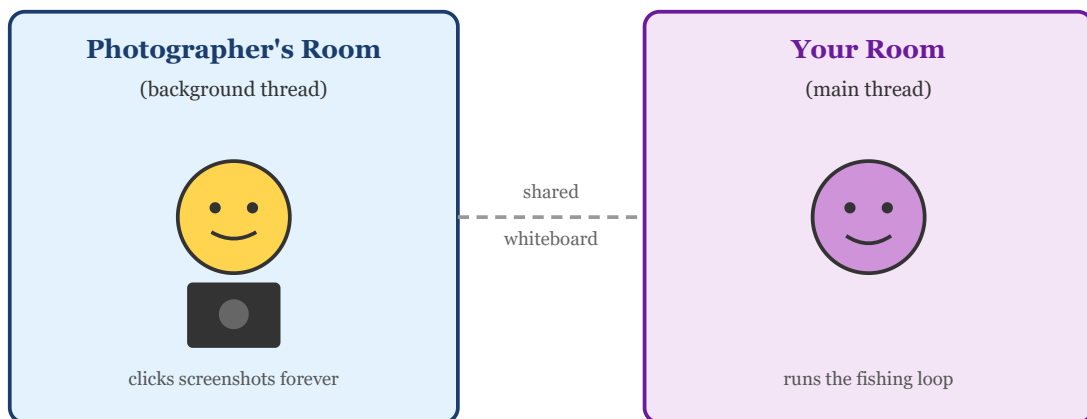
A plain-English guide to the frame capture code, with pictures.

The Big Picture

Your program has **two workers** running at the same time:

- **The Photographer** — a background worker that never stops. Every fraction of a second it takes a fresh screenshot of your fishing game.
- **You (the main program)** — your fishing logic. When it wants to check the water, it says "give me the newest picture."

They live in **separate rooms** but need to share **one whiteboard**. If both try to touch it at the same time, chaos.



Two workers, one shared whiteboard. This is what needs coordinating.

The Three Tools

1. The Whiteboard — `_latest_frame`

```
_latest_frame = None
```

A single spot in memory that holds the newest picture. Starts empty (`None`). The Photographer wipes it and redraws every time a new picture arrives.

2. The Marker — `_frame_lock`

```
_frame_lock = threading.Lock()
```

A marker with only **one cap**. Whoever holds it can write on the whiteboard. Anyone else has to wait in line until the marker is put back.

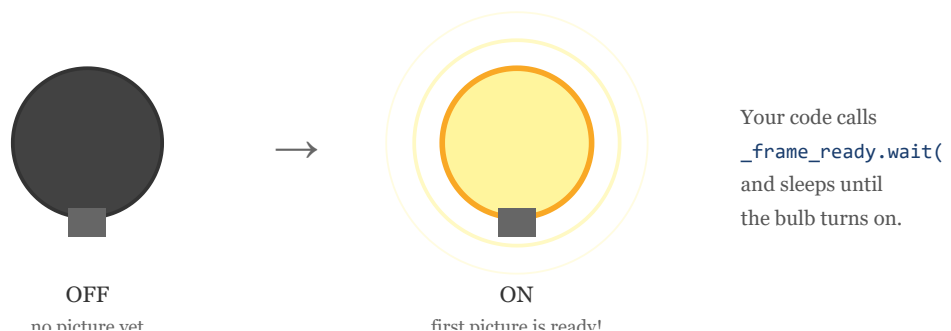


The lock stops both workers from scribbling at once.

3. The Light Bulb — `_frame_ready`

```
_frame_ready = threading.Event()
```

A light bulb that starts **OFF**. It flips **ON** the very first time the Photographer finishes a picture. You can stare at the bulb and wait — Python literally pauses your code until the light turns on.



The event stops you from reading the whiteboard before anything's been drawn on it.

Walking Through the Code

Hiring the Photographer

```
_cap = WindowsCapture(window_name=WIN_NAME)
```

You **hire** a photographer and tell them **which window** to point their camera at. They haven't started yet — this is just the paperwork.

Giving Them the Rulebook

```
@_cap.event
def on_frame_arrived(frame, capture_control):
    global _latest_frame
    with _frame_lock:
        _latest_frame = frame.frame_buffer.copy()
    _frame_ready.set()
```

This is the **instruction sheet** you hand the photographer. It reads:

- **"Every time you take a picture, do these steps..."**
- `global _latest_frame` — "The whiteboard I mean is **the one on the wall**, not a new one you invented."
- `with _frame_lock:` — "Pick up the marker. If someone else has it, wait."
- `_latest_frame = frame.frame_buffer.copy()` — "Wipe the whiteboard. Draw the new picture. **Make a copy** — the original photo goes back in your bag, you leave a copy behind."
- (end of `with` block) — "Put the marker back so someone else can use it."
- `_frame_ready.set()` — "Flip the light bulb ON."

Why the `.copy()` ? The photographer's camera might reuse or delete the original memory a moment later. If you didn't copy, the whiteboard would end up pointing at a photo that no longer exists — like a receipt for a bag someone already threw away.

The Empty Rule

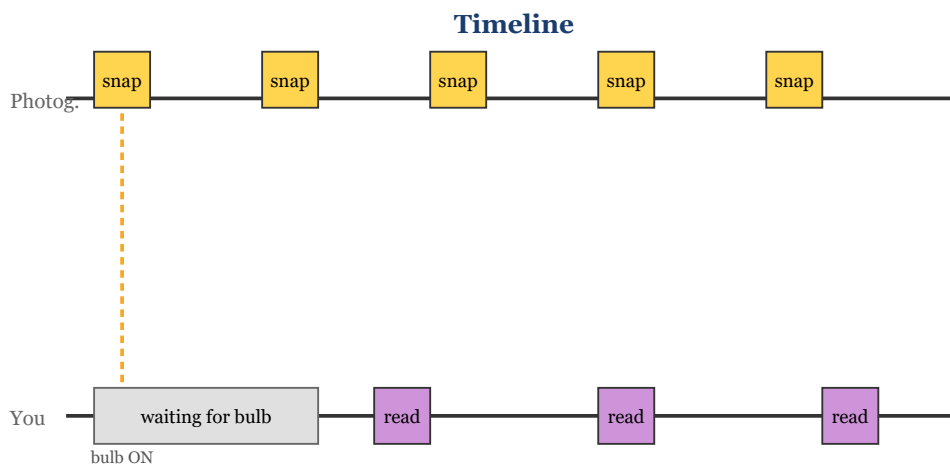
```
@_cap.event
def on_closed():
    pass
```

"If the photographer ever quits, do nothing." The rulebook *requires* a rule for this, so you write an empty one. The program is closing anyway — nothing to clean up.

Start Working!

```
_cap.start_free_threaded()
```

You clap your hands. The photographer walks off to their own room and starts clicking pictures forever. **"Free-threaded"** just means: they work in the background, and **you don't have to wait** — your program keeps running its own logic while they work.



Photographer never stops. You read whenever you feel like it.

The two workers on parallel timelines. You only wait at the start for the first picture.

Why Not Just Skip the Locks?

Without the marker (lock): the photographer might be halfway through drawing a new picture when you look at the whiteboard. You'd see a half-old, half-new mess. In code terms: a corrupted image, a crash, or worse, a subtle bug that only happens sometimes.

Without the light bulb (event): your very first attempt to read might happen **before** any picture exists. The whiteboard would still say `None`. Your code would try to process "nothing" and crash immediately.

With both: your reads are always safe and always see a complete, real picture. The waiting only happens once, at startup. After that, it's smooth sailing.

One-Sentence Summary

A background worker takes screenshots forever; the **lock** makes sure you never see a half-written one, and the **event** makes sure you don't peek before the first one exists.